Практикум по STM8

STM8SVLDISCOVERY

Попов Роман
Май 2012 г.

Структура накопителя

Скопируйте все содержимое накопителя в отдельную папку на вашем ПК

Структура накопителя:

Docs – документация на микроконтроллер, отладочную плату

Exercises – библиотеки, лабораторные работы

Slides – презентации

Tools – среда разработки

Инсталляция IAR и STToolset

Шаг 1:

-Скопируйте содержимое флеш-накопителя в отдельную папку на вашем диске.
- Запустите установку среды по следующему пути X:\Tools\IAR\

Шаг 2:

-Следуйте по шагам установщика. При требовании установщиком лицензионного номера и ключа, скопировать их из файла ReadMe.txt
- Данная версия имеет ограничение по объему загружаемого кода 8Кбайт

Шаг 3:

- Установите также бесплатный инструментарий от ST – STToolset. Он понадобится для работы с EEPROM памятью и Options Bytes

Lab1: Мерцание светодиодом

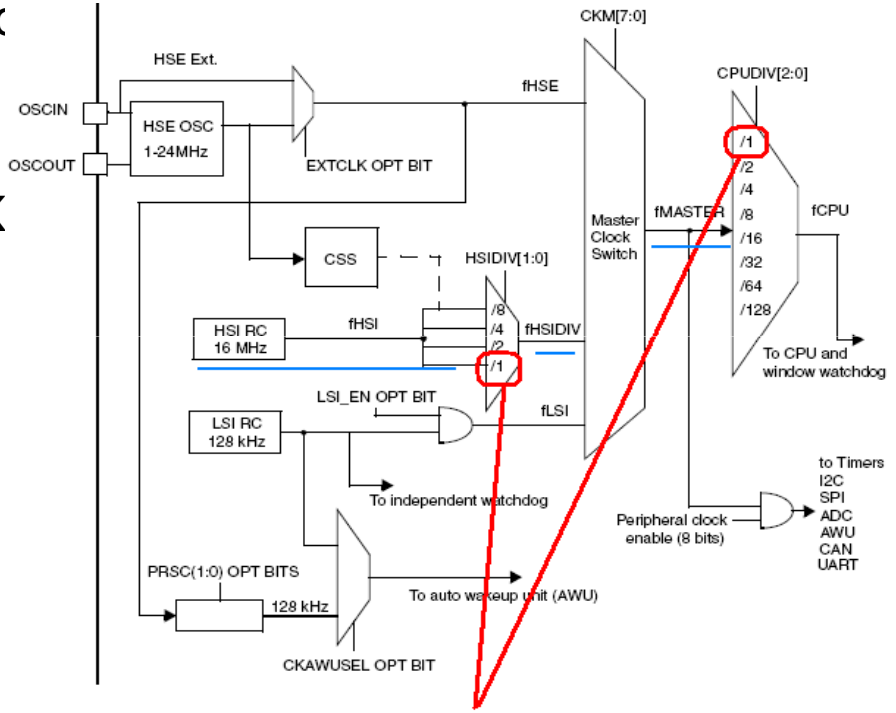
Цель лабораторной работы – получение необходимых знаний о конфигурации тактовой частоты и настройке GPIO:
- **Настройка тактовой частоты микроконтроллера**
- **Настройка порта в/в**

Lab1: Мерцание светодиодом

Часть 1. Настройка тактовой частоты

- После сброса микроконтроллера он работает на частоте 2 МГц от источника HSI/8.
- Вам необходимо настроить рабо 16МГц
- Установка предделителей

производится в регистре **CLK_CK**



Установить преддлители = 1

Lab1: Мерцание светодиодом

Часть 2. Настройка GPIO

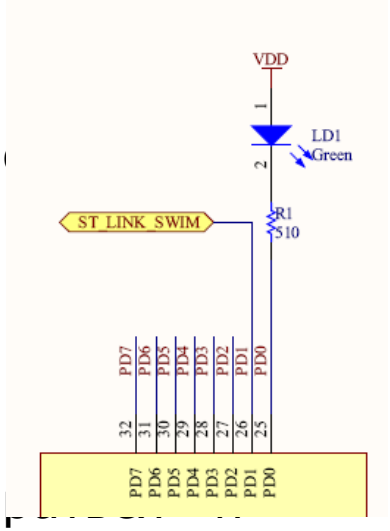
- После **сброса** все ножки в/в в режиме **input floating**
- Ножки сгруппированы в порты по 8 бит (PA,PB,..PH)
- GPIO конфигурируются несколькими 8-бит регистрами:
  - **Px_ODR** – регистр выходных данных
  - **Px_IDR** - регистр входных данных
  - **Px_DDR** - регистр направления данных ножки (0 - вход; 1 - выход)
  - **Px_CR1** – регистр настройки 1 (floating, push-pull, OD зависит от направления вывода)
  - **Px_CR2** – регистр настройки 2 (внешние прерывания или скорость вывода)

Режим конфигурации	DDR	CR1	CR2 / ADCTDR
Analog Input	0	0	CR2 = 0 ADCTDR =1
Input floating	0	0	Работа с внешними прерываниями
Input Pull-up	0	1	
Open-drain output	1	0	Ограничение по скорости 10 МГц
Push-pull Output	1	1	
True open-drain	1	X	X

Lab1: Мерцание светодиодом

Необходимо сделать:

- На отладочной плате имеется светодиод LD1, схема п
- Необходимо ножку PD0 настроить :
  - На выход
  - Режим push-pull
  - Выходной каскад на 10 МГц
- При записи «0» в регистр Px_ODR диод должен «загор
  «тухнуть» в случае записи «1»



Результат работы программы:

В основном цикле программы состояние PD0 меняется на противоположное через программную задержку delay(). Вы можете поставить комментарии перед настройкой тактовой частоты и убедиться в разнице моргания светодиода, свидетельствующей о работе микроконтроллера на 2 и 16 МГц

Lab2: Работа с прерываниями

Цель лабораторной работы – получение необходимых знаний о конфигурации внешних прерываний и таймера:
- **Настройка внешнего прерывания**
- **Настройка таймера 1 (TIM1)**

Lab2: Работа с прерываниями

Часть 1. Настройка EXTI

- По прерыванию от кнопки будет происходить переинициализация значения таймера 1 на увеличение частоты мерцания светодиода в 2 раза
- После **сброса** все прерывания отключены
- В STM8 на каждый порт выделено 1 прерывание (вектор)
- Глобальное отключение/включение прерываний производится командами **asm("sim")** / **asm("rim")**
- Необходимо настроить внешнее прерывание на ножке PB7(синяя кнопка) - отредактировать функцию **GPIO_Init()**:
  - Настроить ножку **PB7** на **вход**, режим **floating input** (регистры Px_DDR, Px_CR1)
  - Настроить прерывание по заднему фронту (регистр Px_CR1)
  - Разрешить прерывание (регистр Px_CR2)

Lab2: Работа с прерываниями

Часть 2. Настройка TIM1

- Необходимо настроить TIM1 для генерации временного интервала в 1 секунду с прерыванием, в котором будет происходить смена состояния светодиода на противоположное
- Необходимо сделать:
  - Настроить таймер – режим счета вверх, прерывание после достижения заданного значения (update), авто-перезагрузка (регистр TIM1_CR1)
  - Разрешить прерывание таймера (регистр TIM1_IER)
  - Установить частоту тактирования TIM1 = 10 КГц (регистр TIM1_PSCRx)
    - Основная частота тактирования = 2МГц, т.е. предделитель TIM1 = 200
  - Рассчитать правильное значение для записи в счетчик таймера (TIM_CNTR1x) и регистр перезагрузки (TIM1_ARRx) из учета, что один тик таймера равен 1/10КГц

Lab2: Работа с прерываниями

Часть 2. Создание обработчиков прерываний (ISR)

- Необходимо создать 2 обработчика прерываний – по заднему фронту PB7 (пользовательская кнопка) и по достижению значения TIM1
- Обработчики прерываний реализуем в отдельном файле «**stm8_it.c**»
- Любой обработчик прерывания должен начинаться с

```
#pragma vector = <номер прерывания>
__interrupt void <имя прерывания>(void)
{
 }
```

Где:

<номер прерывания> - номер прерывания в hex формате. Берется значение из даташита на микроконтроллер и к нему прибавляется 2.

<имя прерывания> - любое имя, понятное вам.

Lab2: Работа с прерываниями

Table 10: Interrupt mapping

IRQ no.	Source block	Description	Wakeup from halt mode	Wakeup from active-halt mode	Vector address
	RESET	Reset	Yes	Yes	0x00 8000
	TRAP	Software interrupt	-	-	0x00 8004
0	TLI	External top level interrupt	-	-	0x00 8008
1	AWU	Auto wake up from halt	-	Yes	0x00 800C
2	CLK	Clock controller	-	-	0x00 8010
3	EXTI0	Port A external interrupts	Yes(1)	Yes(1)	0x00 8014
4	EXTI1	Port B external interrupts	Yes	Yes	0x00 8018
5	EXTI2	Port C external interrupts	Yes	Yes	0x00 801C
6	EXTI3	Port D external interrupts	Yes	Yes	0x00 8020
7	EXTI4	Port E external interrupts	Yes	Yes	0x00 8024
8		Reserved	-	-	0x00 8028
9		Reserved	-	-	0x00 802C
10	SPI	End of transfer	Yes	Yes	0x00 8030
11	TIM1	TIM1 update/ overflow/ underflow/ trigger/ break	-	-	0x00 8034

+2

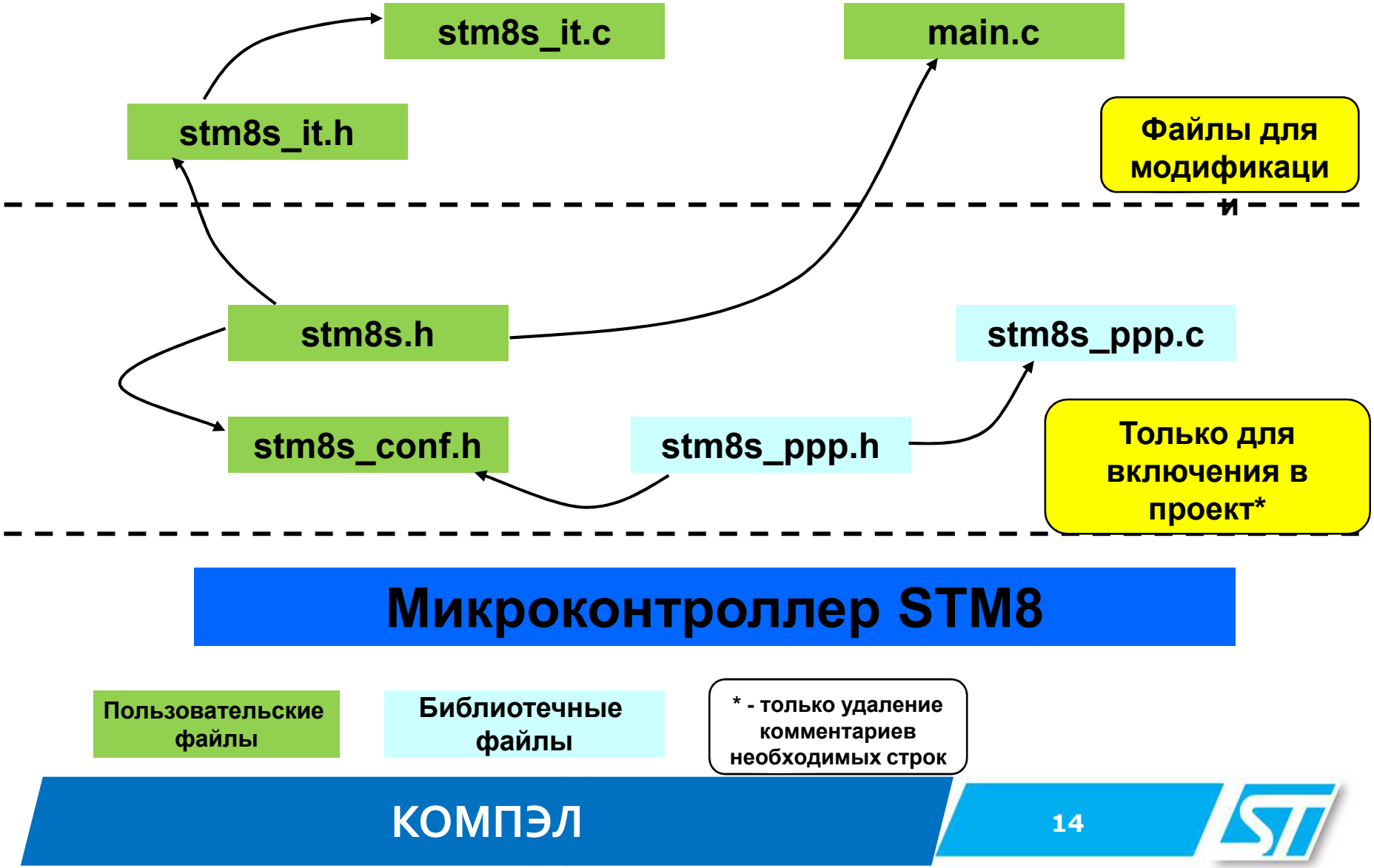
....

Lab2: Работа с прерываниями

Результат работы программы:

Меняющаяся частота мерцания светодиода LD1 в круговом режиме.

Структура библиотеки



Структура файлов библиотеки

Файл	Описание файла
stm8s.h	Файл доступа к библиотеке периферийных устройств. Данный файл необходимо добавить в файл основной программы(обычно «**main.c**»). Содержание файла: 1. Конфигурационная секция для выбора: - Тип микроконтроллера - Использование/неиспользование библиотеки(прямой доступ к регистрам периферии) через макрос «***#define USE_STDPERIPH_DRIVER***» - Конфигурация некоторых специфичных параметров МК, например частота тактирования HSE 2. Структуры данных и отображение адресов на всю периферию 3. Объявление регистров периферии и битовые маски 4. Макросы для доступа к определенным полям регистров периферии
stm8s_conf.h	Конфигурационный файл библиотеки периферии. Пользователь может подключать или отключать заголовочные файлы необходимых периферийных модулей через удаление комментариев с необходимой строки. Также в данном файле можно отключить/подключить проверку входных параметров функций в реальном времени через препроцессорное объявление «**USE_FULL_ASSERT**»
stm8s_ppp.h	Заголовочный файл периферийного модуля **ppp** (SPI, I2C, TIMx и т.д.) Файл включает объявления функции периферии, переменные, макроопределения
stm8s_ppp.c	Реализация функций, объявленных в соответствующем файле «**stm8s_ppp.h**»
stm8s_it.h	Заголовочный файл всех функций – обработчиков прерываний МК
stm8s_it.c	Реализация функций – обработчиков прерываний МК

Использование библиотеки(1/2)

- Библиотека написана на ANSI – C
  - ✓ На ANSI – C написаны все примеры и вся библиотека
- Справка в файле **stm8s-a_stdperiph_lib_um.chm** пакета библиотеки
- Все переопределенные типы данных находятся в файле «**stm8s.h**»
- Термин **PPP** используется для обобщения любого периферийного устройства, например **TIM** для таймера или **GPIO** для портов в/в
- Регистры и структуры
  - ✓ Имена регистров STM8 отражают адресное пространство МК
    - ✓ Регистры библиотека стандартных устройств имеют такие же имена, как в документации/app notes на МК STM8
  - ✓ Доступ ко всем регистрам производится через С структуры:
    - ✓ Все структуры и объявления находятся в файле «**stm8s.h**»
    - ✓ Работа только с одним базовым адресом и косвенной адресацией
    - ✓ Улучшенное повторное использование кода, например одинаковая структура для работы с USART
  - ▪ Доступ к любому регистру осуществляется: SPI1 -> CR1 = 0x01

Использование библиотеки(2/2)

- Для каждого периферийного модуля существует заголовочный файл и файл реализации, где описаны и реализованы все функции для настройки и работы с модулем, например для SPI это «**stm8s_SPI.h**» и «**stm8s_SPI.c**»
- Одинаковые символические имена функций для разных периферийных модулей:
  - ✓ Инициализация модуля «**PPP_Init**» (например **SPI_Init**)
  - ✓ Сброс значений по умолчанию(RESET) «**PPP_DeInit**» (например **TIM1_DeInit**)
  - ✓ Включение/отключение модуля «**PPP_Cmd**» (например **UART1_Cmd)**
  - ✓ Разрешение/запрет прерываний модуля «**PPP_ITConfig**» (например **TIM3_ITConfig)**
  - ✓ Определение флагов модуля «**PPP_GetFlagStatus**» (например **GPIO_GetFlagStatus)**
  - ✓ Сброс флагов модуля «**PPP_ClearFlag**» (например **I2C_ClearFlag)**
  - ✓ Определение возникновения прерывания «**PPP_GetITStatus**» (например **UART3_GetITStatus)**
  - ✓ Сброс флага сработавшего прерывания «**PPP_ ClearITPendingBit**» (например **CLK_ ClearITPendingBit)**
- Осуществляется проверка входных параметров функций через макрос «***assert_param»***
- Пример настройки и конфигурации библиотеки рассмотрен в лабораторной работе №1

Lab3: Создание собственного проекта и конфигурация библиотеки

Цель лабораторной работы – получение необходимых знаний о конфигурации библиотеки периферийных устройств STM8S

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 1:

- Запустите среду IAR EW и создайте новое рабочее поле и проект в этом поле: **File -> New -> Workspace** и **Project -> Create New Project**

Шаг 2:

- Выберите в выпадающем меню проект C – main
- Укажите имя проекта **“Lab3”** по следующему пути
  X:\Exercises\Project\Lab1

Где **X** – имя диска в вашей системе

Шаг 3:

- Сохраните имя рабочей области в папку EWARM вашего проекта: кликните **File -> Save WorkSpace**, и введите имя , например - **“Lab3”**

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 4:

- Создадим структуру проекта в окне **«Workspace»**. В будущих проектах вы можете создавать ее по своему усмотрению. Добавление файлов и папок осуществляются кликом правой кнопкой мыши на имени проекта и кликом на **“Add Group…”** и **“Add Files…”**. Итак, создаем следующие папки
- **StdPeriph_Driver**
- **User**
- Перемещаем основной файл проекта «**main.c**» в папку **“User”**, перетащив его с помощью мыши

Шаг 5:

Для работы с проектом нам необходимо скопировать следующие файлы из шаблона проекта(путь **X:\Exercises\Project\Template**) в созданную нами ранее папку **Lab1** через любой файловый менеджер: **stm8s_it.c, stm8s_it.h** и **stm8s_conf.h.**

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 6:

StdPeriphDriver:

В данной папке будут храниться файлы библиотеки периферийных устройств. По мере необходимости, сюда следует добавлять файлы периферийных устройств(правая кнопка мыши на папке –> Add Files…), с которыми
вы будете работать. Например:

stm8s_clk.c – тактирование
stm8s_gpio.c – порты в/в
stm8s_tim1.c – таймер 1
stm8s_adc2.c – АПЦ2
и т.д.

Мы же добавим сюда файлы **stm8s_clk.c** и **stm8s_gpio.c,** находящихся в **X:\Exercises\Libraries\STM8S_StdPeriph_Driver\src**

User:

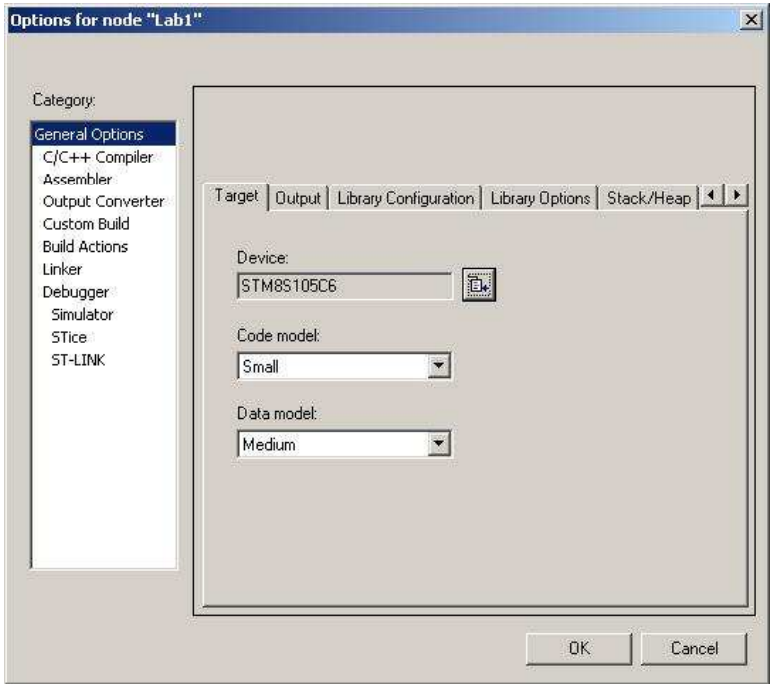
В данной папке будем хранить все написанные нами файлы, добавляем файлы **stm8s_it.c** и **stm8s_conf.h**, лежащих в папке нашего проекта Lab1.

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 7:

На данном этапе нам необходимо настроить опции проекта. Все опции проекта можно посмотреть и настроить следующим образом: клик правой кнопкой мышки на имени проекта и **«Options…»**

1. Выбираем тип МК – STM8S105C6



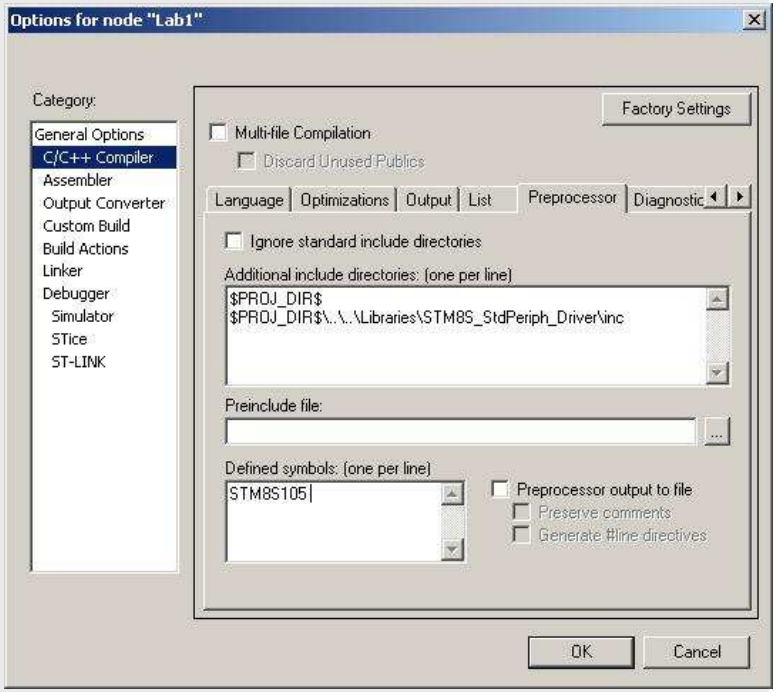
Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 7:

2. Прописываем пути ко всем файлам библиотеки, вкладка «**C/C++ Compiler -> Preprocessor**», поле «**Additional include directories:**». Первая строка указывает путь, где находится файл проекта, в нашем случае это «**X:\Exercises\Project\Lab1**». Вторая строка указывает путь к заголовочным Файлам библиотек:

$PROJ_DIR$ - текущий путь к файлу проекта
$PROJ_DIR$\.. - на один уровень вверх по папкам проекта

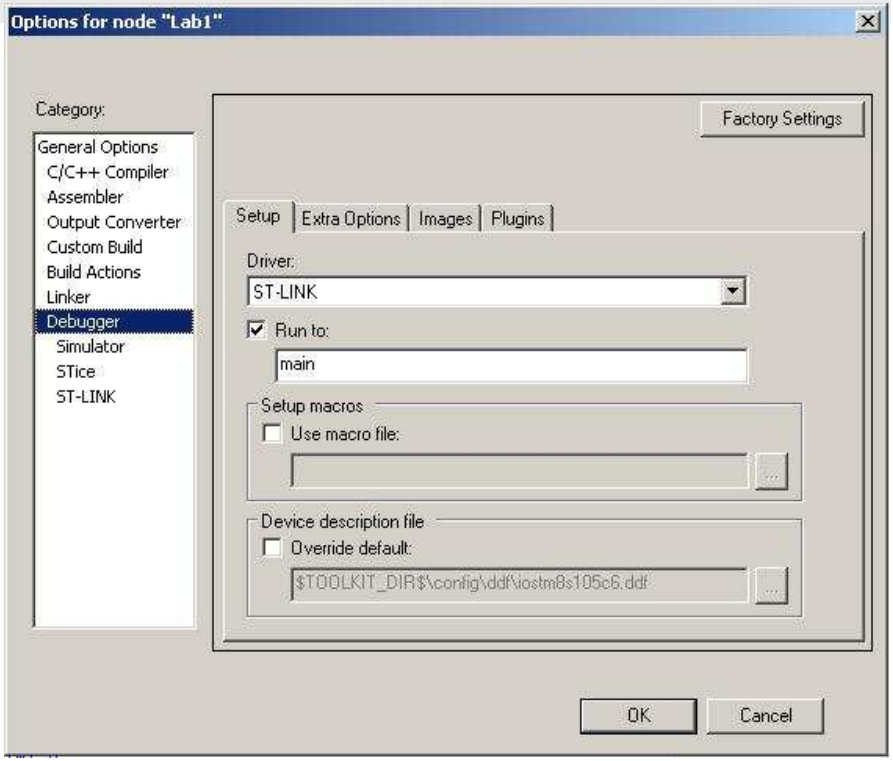
Пути можно скопировать из файла **ReadMe.txt,** находящегося в папке Lab3.



Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 7:

4. Во вкладке «**Debugger -> Setup**», в поле «**Driver**» необходимо указать, что мы будем использовать программатор-отладчик **ST-LINK**



Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 8:

На данном шаге нам необходимо создать тестовый проект и проверить, что все работает правильно. Сейчас нашей целью является проверка работоспособности проекта, то для этого необходимо заменить автоматически сгенерированный код IAR файла «**main.c**» заменить на код из файла «**ReadMe.txt**», пункт 2. Для дальнейшего использования библиотеки необходимо:

1. Включить в начало файла «**main.c**» строку «**#include "stm8s.h"**»
2. Удалить комментарий напротив строки используемого нами микроконтроллера в файле «**stm8s.h**»:

```
/* #define STM8S208 */     /*!< STM8S High density devices with CAN */
/* #define STM8S207 */     /*!< STM8S High density devices without CAN */
/* #define STM8AF52Ax */   /*!< STM8A High density devices with CAN */
/* #define STM8AF62Ax */   /*!< STM8A High density devices without CAN */
 /*  #define STM8S105 */     /*!< STM8S Medium density devices */
/* #define STM8AF626x */   /*!< STM8A Medium density devices */
/* #define STM8S103 */     /*!< STM8S Low density devices */
/* #define STM8S903 */     /*!< STM8S Low density devices */
```

3. Убедиться в том, что в файле «**stm8s.h**» отсутствует комментарий напротив строки: «**#define USE_STDPERIPH_DRIVER**»:

```
#if !defined  USE_STDPERIPH_DRIVER
/* Comment the line below if you will not use the peripherals drivers.
   In this case, these drivers will not be included and the application code will be
   based on direct access to peripherals registers */
 #define USE_STDPERIPH_DRIVER
#endif
```

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 8:

4. Удалить комментарии напротив используемых периферийных модулей в файле «stm8s_conf.h». Также исходные файлы этих модулей необходимо добавить проект, что мы проделали на Шаге 6. В данной лабораторной работе это модули тактирования и портов в/в:

```
/* Uncomment the line below to enable peripheral header file inclusion */
#if defined(STM8S105) || defined(STM8S103) || defined(STM8S903) || defined (STM8AF626x)
//#include "stm8s_adc1.h"
#endif /* (STM8S105) ||(STM8S103) || (STM8S903) || STM8AF626x*/
#if defined(STM8S208) || defined(STM8S207) || defined (STM8AF52Ax) || defined (STM8AF62Ax)
// #include "stm8s_adc2.h"
#endif /* (STM8S208) || (STM8S207) || (STM8AF62Ax) || (STM8AF52Ax) */
//#include "stm8s_awu.h"
//#include "stm8s_beep.h"
#if defined (STM8S208) || defined (STM8AF52Ax)
//#include "stm8s_can.h"
#endif /* STM8S208 || STM8AF52Ax */
#include "stm8s_clk.h"
//#include "stm8s_exti.h"
//#include "stm8s_flash.h"
#include "stm8s_gpio.h"
//#include "stm8s_i2c.h"
//#include "stm8s_itc.h"
//#include "stm8s_iwdg.h"
//#include "stm8s_rst.h"
//#include "stm8s_spi.h"
//#include "stm8s_tim1.h"
```

Lab3: Создание собственного проекта и конфигурация библиотек

Шаг 9:

Собираем проект – **Project – Rebuild All,** все должно собраться и слинковаться без ошибок и предупреждений

Lab4: Мерцание светодиодом на основе библиотек

Lab4: мерцание светодиодом на основе библиотек

Цель лабораторной работы – получение необходимых знаний о конфигурации тактовой частоты и настройке GPIO:

- **Настройка тактовой частоты микроконтроллера**
- **Настройка порта в/в**

Lab4: Мерцание светодиодом на основе библиотек

Цель данной лабораторной работы – научиться пользоваться библиотекой периферийных устройств, на примере конфигурации портов в/в и тактовой частоты микроконтроллера

Создавать проект уже не требуется, необходимо открыть уже созданный проект по пути **X:\Exercises\Project\Lab4** и отредактировать необходимые файлы

- ***Необходимо отредактировать файл main.c:***

По умолчанию, после сброса МК он тактируется от встроенного источника HSI/8 = 2МГц.
1.Необходимо установить тактовую частоту HSI на максимальную – 16МГц(предделитель = 1)
2.Настроить GPIO – порт D0, к нему присоединён светодиод LD1, притом он аппаратно притянут внешним резистором к напряжению питания, посему нам необходимо настроить ножку как push-pull на вход, режим порта – 2МГц.
3.Далее в цикле основной программы необходимо сделать инверсию ножки, т.е. заставить светодиод мерцать.

Lab4: Мерцание светодиодом на основе библиотек

Результатом работы программы должен являться мерцающий светодиод с примерной частотой 1 сек.

Можете поэкспериментировать с предделителями в функции «**CLK_HSIPrescalerConfig()**», частота мерцания светодиода должна меняться.

Lab3: Использование TIM3 для мерцания светодиода

Lab3: Использование TIM3 для мерцания светодиода

Цель лабораторной работы конфигурация TIM3 и мерцание светодиодом по прерыванию от таймера:
- Настройка GPIO
- Настройка TIM3
- Настройка прерывания по переполнению TIM3

Lab3: Использование TIM3 для мерцания

Открываем проект по пути **X:\Exercises\Project\Lab3**

Часть1: Конфигурация GPIO

Тут все просто, просьба не подсматривать в лабораторную работу Lab2 :)

Часть2: Конфигурация TIM3

Тактовая частота подаваемая на TIM3 = 2МГц. Установить предделитель таймера = 32 и значение инициализации счетчика таймера(заранее посчитанное значение в макросе «PERIOD_1s» в начале файла **main.c**)

Часть 3: Настройка прерывания

Разрешить прерывание по переполнению таймера, далее необходимо разрешить глобальное разрешение прерываний. Необходимо определить номер вектора обработчика прерываний и вставить в этот обработчик прерывания закомментированные строки из файла **main.c:**

TIM3_ClearITPendingBit(TIM3_IT_UPDATE);

GPIO_WriteReverse(GPIOD, GPIO_PIN_0);

Все обработчики прерывания реализованы в **stm8s_it.c**

Lab3: Использование TIM3 для мерцания светодиода

Результатом работы будет мерцание светодиода с частотой 1 секунда, можно расставить точки останова и удостовериться, что прерывание работает

Lab5: Работа с UART на основе библиотек

Цель лабораторной работы конфигурация и обмен данными
По интерфейсу UART2:
- Настройка GPIO
- Настройка UART2
- Настройка прерывания по приему байта от UART2

Lab5: Работа с UART на основе бибиотек

Открываем проект по пути **X:\Exercises\Project\Lab4**

Часть1: Конфигурация GPIO

Тут все просто, просьба не подсматривать в лабораторную работу Lab2 :)

Часть2: Конфигурация UART2

Необходимо настроить модуль в соответствии с комментариями, приведенными в основном листинге программы. В основном цикле программы с задержкой передается байт 0x35. Для приема байта вам необходимо с помощью перемычки замкнуть контакты PD5 и PD6, схему STM8S-Discovery можно посмотреть в **X:\Docs\Discovery_Sheme.pdf**

Часть 3: Настройка прерывания

Вам необходимо разрешить прерывание по приему байта и вставить необходимые строки в обработчик прерывания(закомментированы в файле main.c), все обработчики прерывания реализованы в **stm8s_it.c.**

Lab5: Работа с UART на основе библиотек

Обработчик прерывания:

```
 if(UART2_ReceiveData8() == 0x35)
    GPIO_WriteReverse(GPIOD, GPIO_PIN_0);
   else
    GPIO_WriteLow(GPIOD, GPIO_PIN_0);

   UART2_ClearITPendingBit(UART2_IT_RXNE);
```

Результат работы программы:

В основном цикле программы отправляется байт 0x35. В обработчике прерывания происходит проверка принятого байта. В случае если принятый байт равен значению 0x35, то по принятию каждого байта будет мерцать светодиод(частота следования байт ~ 1сек). Если байт принят неверно или вообще не принят, светодиод будет оставаться в статичном положении.

Соедините и разъедините контакты 7 и 6 разъема CN4 перемычкой и убедитесь, что байты принимаются верно – светодиод должен моргать.

Lab6: АЦП на основе библиотек

Цель лабораторной работы научиться работать с АЦП:
-Настройка параметров АЦП
- Измерение сигнала с помощью АЦП

Lab6: АЦП на основе библиотек

Введение

Необходимо настроить работу АЦП в непрерывном режиме сканирования первых двух каналов – 0 и 1

Часть 1 – настройка параметров GPIO

- Настроить тактовую частоту микроконтроллера на 16МГц
- Настроить два канала АЦП (AIN0 и AIN1) для ножек PB0 и PB1 – режим плавающий, без внешних прерываний

Lab6: АЦП на основе библиотек

Часть 2 – настройка параметров АЦП

- Включить режим сканирования (scan)
- Параметры АЦП:
  - Режим работа – continuous
  - Установить канал1. В сканирующем режиме мы проставляем номер последнего канала, до которого будет идти сканирование, после него автоматически начнется преобразование нулевого канала
  - Установить тактовую частоту модуля = 4МГц. Максимальное время преобразования достигается при тактовой частоте 6МГц, ее возможно получить только от внешнего кварца
  - Триггер таймера(внешний источник запуска) отключить
  - Выравнивание по правому краю
  - Триггер Шмидта можно отключить

Lab6: АЦП на основе библиотек

Результат работы

В данном случае производится поочередное преобразование двух каналов АЦП без прерывания. В основном цикле программы просматривается результат измерения, и как только он становится порядка 3В, загорается светодиод LD1.

Для проверки можно перемычкой соединить каждый канал с «землей» (светодиод гаснет) или напряжением питания (светодиод загорается). Результаты измерения также можно посмотреть через останов программы и просмотра переменной ADC_res

Lab7: ШИМ сигнал на таймере

Цель лабораторной работы научиться работать с TIM2:
- Настройка параметров ШИМ сигнала

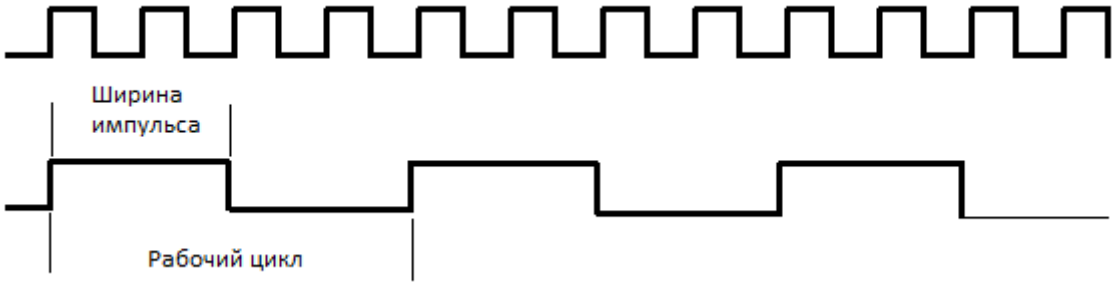
Lab7: ШИМ сигнал на таймере

Введение

Необходимо настроить TIM2 на выдачу на ножку микроконтроллера PC1 сигнала с частотой 16 КГц, рабочий цикл – 50%. Далее можно «поиграться» с рабочим циклом.

Часть 1 – настройка параметров таймера

- Тактовая частота, подаваемая на таймер = 16 МГц, необходимо настроить параметры выходного сигнала – частота сигнала 16КГц и рабочий цикл – 50%:
  - Ширина импульса задается в регистре - TIM2_CCR1
  - Рабочий цикл задается в регистре - TIM2_ARR



Lab7: ШИМ сигнал на таймере

- Сигнал можно посмотреть с помощью осциллографа на ножке PD4 – первый канал TIM2

Lab8: Работа с EEPROM на основе библиотек

Цель лабораторной работы конфигурация, считывание и запись данных во внутреннюю EEPROM память микроконтроллера:
- Настройка параметров FLASH/EEPROM
- Побайтная запись/чтение в EEPROM
- Побайтная очистка EEPROM

Lab8: Работа с EEPROM на основе библиотек

Введение.

EEPROM и FLASH управляются общим набором регистров, поэтому работа с ними фактически идентичная. Организация памяти элементами по 4 байта каждый. Запись можно производить байтами, словами и блоками.

Работа состоит из двух частей – стирание и запись байта по определенному адресу. Данные части разделены в программе директивами условной компиляции

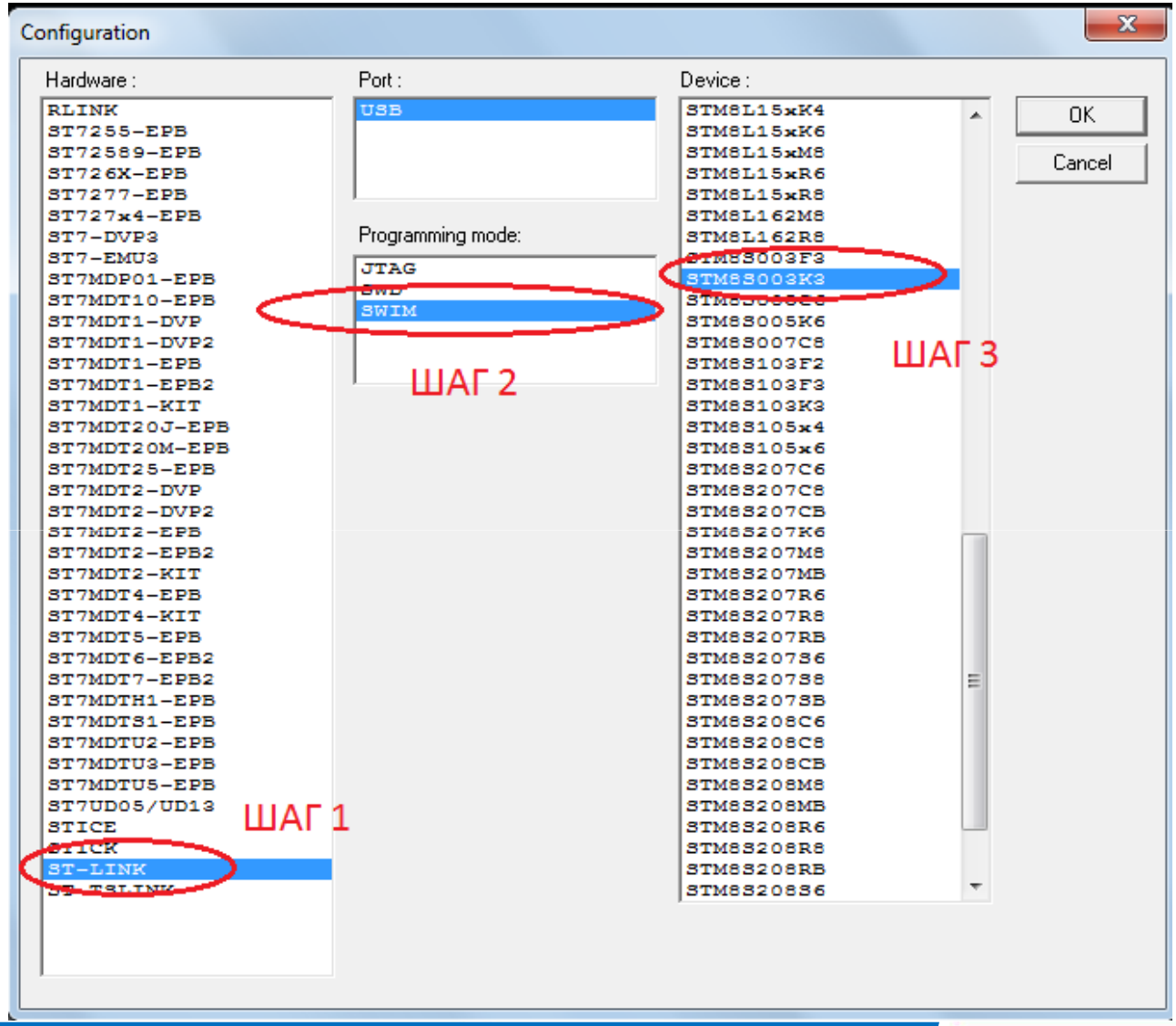
Часть 1. Запись байта

- Удалите комментарий напротив строки: *#define PART1,* остальные строчки (PARTx) должны быть закомментированы
- В начале работы необходимо установить параметры для работы с памятью – время программирования памяти. Мы устанорвим быстрое время = ½ tprog
- После сброса, защита памяти автоматически активирована, ее необходимо снять путем записи специальных ключей (MASS)

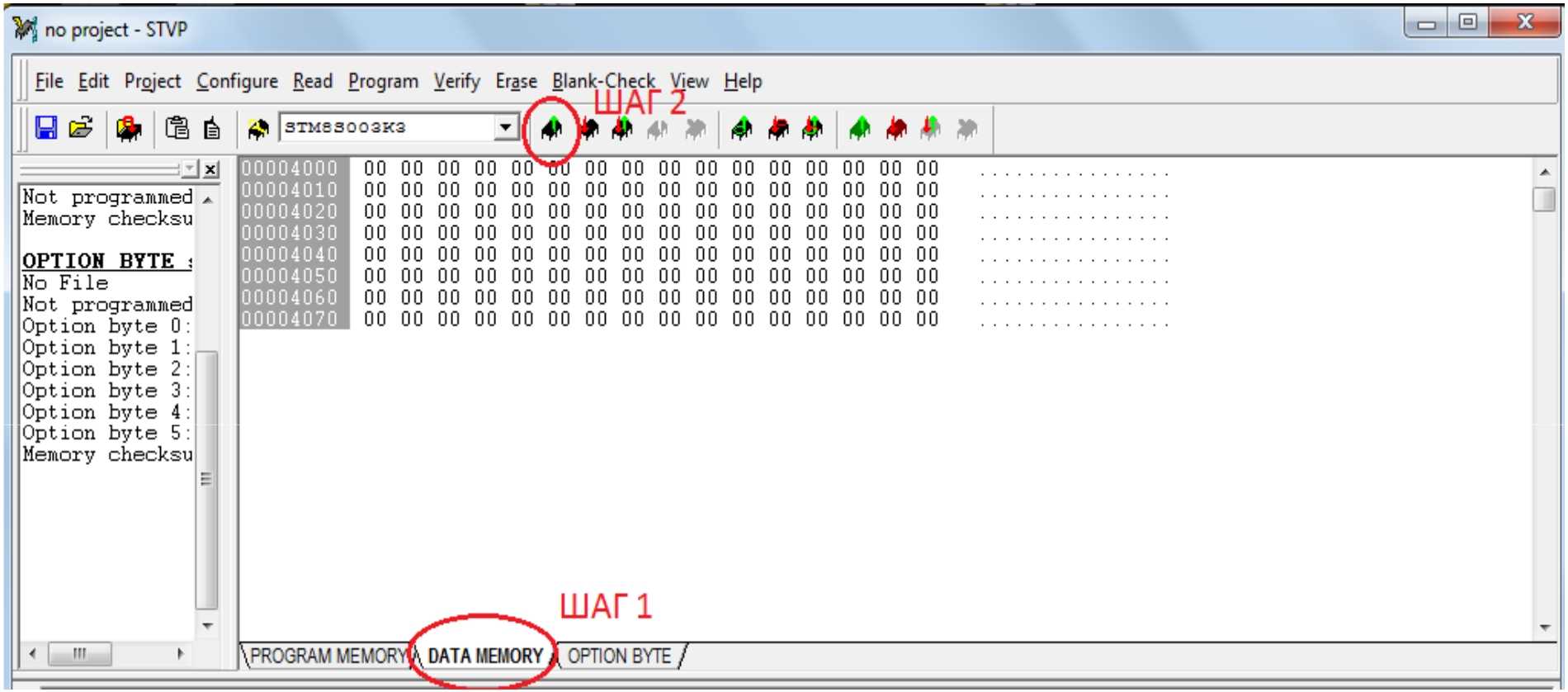
Lab8: Работа с EEPROM на основе библиотек

- Начальный адрес EEPROM во всех микроконтроллерах STM8S - 0x4000, конечный зависит от типа микроконтроллера, вычисляется простым смещением от начального адреса объема EEPROM памяти. Запишем
- Запишем байт по начальному адресу, это производится функцией *FLASH_ProgramByte(X, Y),* где X – адрес, Y – Значение. Запишите любое число, например 0xAA.
- Чтение производится функцией *FLASH_ReadByte(X)*, где X – адрес, по которому записана переменная. Если чтение будет выполнено не верно – не по тому адресу, или байт записан ошибочно, светодиод на плате будет мерцать. Если все выполнено верно, то он будет гореть постоянно.
- Посмотреть, что записано в EEPROM можно в программе ST Visual Programmer:

Lab8: Работа с EEPROM на основе библиотек



Lab8: Работа с EEPROM на основе библиотек



Lab8: Работа с EEPROM на основе библиотек

Часть 2. Очистка байта

- Удалите комментарий напротив строки: *#define PART2,* остальные строчки (PARTx) должны быть закомментированы
- Т.к. данный фрагмент кода компилируется заново, без предыдущего, то все операции по снятию защиты, установки времени программирования необходимо провести заново
- Стирание байта производится функцией *FLASH_EraseByte(X)*, где X – адрес переменной.
- Если процесс очистки прошел правильно, то светодиод LD1 на плате должен гореть постоянно. Также, можно убедиться в том, что байт стерт правильно в программе ST Visual Programmer, только ее придется запустить заново.